GUIDE PAS À PAS – PROJET 2

Construire un agent IA avec outils, puis lui donner une interface web comme ChatGPT

Python, Azure OpenAI, appel de fonctions (function calling), Flask, HTML/CSS/JavaScript

PARTIE A – L'agent

Un agent qui choisit seul ses 3 outils

Outils Python, description JSON des outils, boucle de l'agent, rôle « tool », garde-fou, exercices.

PARTIE B – Le TP web

Une interface de chat dans le navigateur

Serveur Flask, API /api/chat, page HTML/CSS/JavaScript, historique, sécurité (XSS, rôles), grille d'évaluation.

Public : débutants en Python et en IA générative. Chaque étape est numérotée, avec le code complet, l'explication ligne par ligne, le résultat attendu, les erreurs fréquentes et des exercices.

Fichiers : le code de ce guide est identique à celui du dossier `projet2-agent/` du zip de la formation.

Les clients cités (Banque Horizon, Energia Nord, Groupe Hospitalier Saint-Lumière), leurs budgets et les taux de change sont fictifs, créés pour la formation.

Sommaire

Sommaire 2

0. Avant de commencer 3

0.1 Ce qu'il faut sur le poste . . . 3

0.2 Les notions à connaître . . . 3

0.3 RAG ou agent : quelle différence ? . . . 3

Partie A – Construire l'agent avec 3 outils 4

Étape 1 – Comprendre le fonctionnement d'un agent . . . 4

Étape 2 – Préparer le projet . . . 4

Étape 3 – Se connecter à Azure OpenAI . . . 6

Étape 4 – Écrire les 3 outils (de simples fonctions Python) . . . 6

Étape 5 – Décrire les outils au LLM . . . 7

Étape 6 – La boucle de l'agent : le cœur du projet . . . 8

Étape 7 – Le programme dans le terminal . . . 10

Étape 8 – Dépannage et exercices . . . 11

Partie B – TP : une interface web comme ChatGPT 12

Étape 9 – L'architecture : trois acteurs . . . 13

Étape 10 – Organiser les fichiers et installer Flask . . . 14

Étape 11 – Le serveur : app.py . . . 14

Étape 12 – La page web : index.html . . . 15

Étape 13 – Lancer et tester l'application . . . 19

Étape 14 – Dépannage du TP . . . 21

Étape 15 – Pour aller plus loin, et évaluation du TP . . . 22

Annexes 24

Annexe 1 – Récapitulatif des commandes . . . 24

Annexe 2 – Glossaire . . . 24

0. Avant de commencer

0.1 Ce qu'il faut sur le poste

Outil	Pourquoi	Vérification
Python 3.10 ou plus récent	Le langage du projet	`python --version`
Visual Studio Code + extension « Python »	L'éditeur utilisé dans la formation	Ouvrir VS Code → Extensions
Un navigateur (Edge, Chrome, Firefox)	Pour le TP de l'interface web (partie B)	—
Un accès Azure OpenAI	Un endpoint, une clé API et un **déploiement** de modèle de chat	Azure AI Foundry → Déploiements

0.2 Les notions à connaître

Notion	En une phrase
LLM	Un modèle de langage : il lit du texte et produit la suite la plus probable. Il ne peut ni consulter une base de données, ni connaître la date, ni faire un calcul fiable par lui-même.
Rôles des messages	`system` (consignes), `user` (question), `assistant` (réponse du modèle), `tool` (résultat d'un outil). Ils s'écrivent en minuscules.
Outil (tool)	Une fonction Python que le LLM peut **demander** d'exécuter. Il ne l'exécute jamais lui-même : c'est notre code qui le fait.
Agent	Un LLM + des outils + une boucle : le LLM décide quels outils appeler, combien de fois, puis répond.
Le LLM n'a pas de mémoire	À chaque appel, on lui renvoie toute la conversation. C'est notre code qui garde l'historique dans la liste `messages`.

0.3 RAG ou agent : quelle différence ?

	RAG (projet 1)	Agent (projet 2)
Qui décide des étapes ?	Notre code : toujours « chercher, puis répondre »	Le LLM, selon la question
Nombre d'étapes	Une recherche, une réponse	Plusieurs, en boucle
Ce que le LLM peut faire	Lire des documents et répondre	Lire, calculer, consulter, **agir**
Exemple	« Que dit la doc sur le VPN ? »	« Budget de Banque Horizon en dollars ? » → 2 outils

Partie A – Construire l'agent avec 3 outils

Objectif : construire un assistant commercial qui répond à des questions sur des clients fictifs d'Atos. Pour cela, il dispose de trois outils : consulter la fiche d'un client, convertir un montant en devise, et connaître la date du jour. Le point clé : **c'est le LLM qui choisit** les outils à utiliser.

Étape 1 – Comprendre le fonctionnement d'un agent

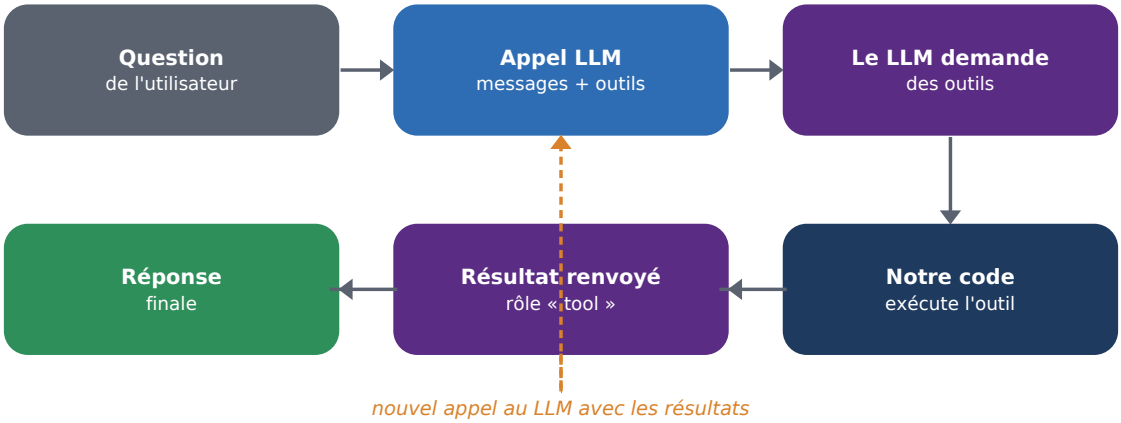


Figure 1 – La boucle de l'agent : elle tourne tant que le LLM demande des outils

Prenons la question « Quel est le budget de Banque Horizon en dollars ? ». Voici ce qui se passe réellement :

1. Notre code envoie au LLM la question **et la description des 3 outils**.
2. Le LLM ne connaît pas le budget. Il répond : « appelle `infos_client` avec `nom=Banque Horizon` ».
3. Notre code exécute la fonction Python et renvoie le résultat au LLM (budget : 4 200 000 €).
4. Le LLM répond : « appelle `convertir_euros` avec `montant=4200000, devise=USD` ».
5. Notre code exécute la conversion et renvoie le résultat.
6. Le LLM a tout ce qu'il lui faut : il rédige la réponse finale en français.

> **i Ce qu'il faut retenir**
> Aucune ligne de notre code ne dit « pour un budget en dollars, appelle infos_client puis convertir_euros ». Le LLM déduit cet enchaînement uniquement à partir des **descriptions** des outils.

Étape 2 – Préparer le projet

Ouvre le dossier `projet2-agent` dans VS Code (**Fichier → Ouvrir le dossier**), puis le terminal (**Terminal → Nouveau terminal**).

2.1 Windows (PowerShell)

```
python -m venv .venv
.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install -r requirements.txt
Copy-Item .env.example .env
```

2.2 macOS / Linux

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
pip install -r requirements.txt
cp .env.example .env
```

Commande	Ce qu'elle fait
`python -m venv .venv`	Crée un environnement virtuel : un Python isolé propre au projet. `-m` exécute le module venv.
`Activate.ps1` / `activate`	Active l'environnement : `(.venv)` apparaît au début de la ligne.
`pip install -r requirements.txt`	Installe les bibliothèques listées dans le fichier (`-r` = « requirement file »).
`Copy-Item .env.example .env`	Crée le fichier des secrets à partir du modèle.

> ⚠ **PowerShell refuse d'activer l'environnement ?**
> Exécuter une fois `Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned`, puis relancer.

2.3 Les dépendances

```
openai>=1.60
python-dotenv>=1.0
flask>=3.0
```

Bibliothèque	Rôle
`openai`	Le client officiel pour appeler Azure OpenAI.
`python-dotenv`	Lit le fichier .env.
`flask`	Le mini serveur web utilisé dans le TP (partie B).

2.4 Le fichier .env

```
# Copier ce fichier en .env puis remplir les valeurs. Le fichier .env n'est JAMAIS commité.
AZURE_OPENAI_ENDPOINT=https://<ta-ressource>.openai.azure.com
AZURE_OPENAI_API_KEY=<ta-cle-api>
# Nom du DÉPLOIEMENT (pas du modèle) créé dans Azure AI Foundry
CHAT_DEPLOYMENT=gpt-5.4-mini-2
```

Remplis les trois valeurs (portail Azure → ta ressource → **Clés et point de terminaison** ; Azure AI Foundry → **Déploiements** pour le nom). Le fichier .gitignore empêche d'envoyer ce fichier sur Git :

```
.env
.venv/
__pycache__/
```

> ⚠ **Modèle ou déploiement ?**
> `CHAT_DEPLOYMENT` doit contenir le **nom du déploiement** que tu as créé dans Azure, pas le nom du modèle. Sinon : erreur 404 DeploymentNotFound.

Étape 3 – Se connecter à Azure OpenAI

Début du fichier (agent.py, lignes 1 à 17)

```
 1  # agent.py
 2  # Agent simple avec 3 outils (tools).
 3  # C'est le LLM qui décide quel outil appeler ; notre code se contente de l'exécuter.
 4  import json
 5  import os
 6  from datetime import datetime
 7
 8  from dotenv import load_dotenv
 9  from openai import OpenAI
10
11  # ---------- Connexion à Azure OpenAI ----------
12  load_dotenv()
13  client = OpenAI(
14      base_url=os.environ["AZURE_OPENAI_ENDPOINT"].rstrip("/") + "/openai/v1/",
15      api_key=os.environ["AZURE_OPENAI_API_KEY"],
16  )
17  deployment = os.environ["CHAT_DEPLOYMENT"]
```

Lignes	Explication
1–3	Les commentaires d'en-tête : ce que fait le fichier.
4–9	Les imports : `json` pour lire les arguments envoyés par le LLM, `datetime` pour la date, `OpenAI` pour le client.
12	Charge le fichier .env dans les variables d'environnement.
13–15	Le client. Le suffixe `/openai/v1/` utilise l'API v1 d'Azure, compatible OpenAI. `rstrip("/")` évite un double « / ».
17	Le nom du déploiement, utilisé à chaque appel.

Étape 4 – Écrire les 3 outils (de simples fonctions Python)

Les outils (agent.py, lignes 20 à 56)

```
 20  # ---------- 1. Les 3 outils : de simples fonctions Python ----------
 21  # Données fictives, pour l'exercice
 22  CLIENTS = {
 23      "banque horizon": {"secteur": "Banque", "ville": "Lyon", "projet": "Migration cloud + SOC 24/
7", "budget_euros": 4200000},
 24      "energia nord": {"secteur": "Énergie", "ville": "Lille", "projet": "Maintenance prédictive
IoT", "budget_euros": 2800000},
 25      "saint-lumière": {"secteur": "Santé", "ville": "Bordeaux", "projet": "Dossier patient
informatisé", "budget_euros": 3500000},
 26  }
 27
 28  # Taux figés pour l'exercice (1 euro = ...)
 29  TAUX = {"USD": 1.08, "GBP": 0.85, "XOF": 655.957, "MAD": 10.9}
 30
 31
 32  def infos_client(nom):
 33      """Outil 1 : renvoie la fiche d'un client."""
 34      for cle, fiche in CLIENTS.items():
 35          if cle in nom.lower() or nom.lower() in cle:
 36              return json.dumps(fiche, ensure_ascii=False)
 37      return "Client inconnu. Clients connus : " + ", ".join(CLIENTS)
 38
 39
 40  def convertir_euros(montant, devise):
 41      """Outil 2 : convertit un montant en euros vers une autre devise."""
 42      devise = devise.upper()
 43      if devise not in TAUX:
 44          return "Devise inconnue. Devises possibles : " + ", ".join(TAUX)
 45      return f"{montant} EUR = {round(montant * TAUX[devise], 2)} {devise}"
 46
 47
```

```
 48  def date_du_jour():
 49      """Outil 3 : renvoie la date d'aujourd'hui (le LLM ne la connaît pas)."""
 50      return datetime.now().strftime("%d/%m/%Y")
 51
 52
 53  OUTILS = {
 54      "infos_client": infos_client,
 55      "convertir_euros": convertir_euros,
 56      "date_du_jour": date_du_jour,
```

Lignes	Explication
22	Une « base de données » fictive : un dictionnaire dont les clés sont les noms des clients en minuscules. En entreprise, ce serait un CRM ou une base SQL.
29	Des taux de change figés pour l'exercice (ce ne sont pas des taux réels, sauf le franc CFA dont la parité avec l'euro est fixe : 655,957).
32–37	**Outil 1.** On cherche le client sans tenir compte des majuscules. `cle in nom.lower()` accepte « Banque Horizon » comme « la banque horizon de Lyon ». `json.dumps(..., ensure_ascii=False)` transforme la fiche en texte, en gardant les accents.
37	Si le client n'existe pas, on renvoie la liste des clients connus : le LLM pourra réessayer avec le bon nom. **Un bon outil explique ses erreurs.**
40–45	**Outil 2.** Le LLM est mauvais en calcul : on lui fournit une fonction exacte. Même principe pour une devise inconnue.
48–50	**Outil 3.** Le LLM ne connaît pas la date du jour : il doit la demander.
53–56	Un « annuaire » : à partir du **nom** d'outil renvoyé par le LLM (un texte), on retrouve la **fonction** Python à exécuter.

> **✓ Tester un outil seul**
> Avant de le donner au LLM, teste chaque outil dans le terminal Python :
> `python` puis `from agent import infos_client` puis `infos_client("Energia Nord")`

Étape 5 – Décrire les outils au LLM

Le LLM ne voit jamais notre code Python. Il ne voit qu'une **description** au format JSON (le format « JSON Schema »). C'est cette description qui lui permet de choisir le bon outil et de remplir les arguments.

La description des outils (agent.py, lignes 60 à 87)

```
 60  # ---------- 2. La description des outils, envoyée au LLM ----------
 61  # Le LLM ne voit pas le code : il ne voit que ces descriptions.
 62  DESCRIPTIONS = [
 63      {"type": "function", "function": {
 64          "name": "infos_client",
 65          "description": "Donne la fiche d'un client d'Atos : secteur, ville, projet, budget en
euros.",
 66          "parameters": {
 67              "type": "object",
 68              "properties": {"nom": {"type": "string", "description": "Nom du client"}},
 69              "required": ["nom"],
 70          },
 71      }},
 72      {"type": "function", "function": {
 73          "name": "convertir_euros",
 74          "description": "Convertit un montant en euros vers USD, GBP, XOF ou MAD.",
 75          "parameters": {
 76              "type": "object",
 77              "properties": {
 78                  "montant": {"type": "number", "description": "Montant en euros"},
 79                  "devise": {"type": "string", "description": "Devise cible, ex : USD"},
 80              },
```

```
 81              "required": ["montant", "devise"],
 82          },
 83      }},
 84      {"type": "function", "function": {
 85          "name": "date_du_jour",
 86          "description": "Donne la date d'aujourd'hui.",
 87          "parameters": {"type": "object", "properties": {}},
```

Champ	Rôle	Conseil
`"type": "function"`	Indique qu'il s'agit d'une fonction appelable.	Toujours identique.
`"name"`	Le nom que le LLM renverra pour demander l'outil.	Doit être identique à la clé du dictionnaire `OUTILS`.
`"description"`	**Le champ le plus important** : le LLM le lit pour décider quand utiliser l'outil.	Dire ce que fait l'outil **et** ce qu'il renvoie.
`"parameters"`	La liste des arguments, avec leur type (string, number…) et leur description.	Décrire le format attendu (ex : « Devise cible, ex : USD »).
`"required"`	Les arguments obligatoires.	Lister tous ceux sans valeur par défaut.

> **⚠ Le nom doit correspondre exactement**
> Si la description dit `"infos_client"` mais que `OUTILS` contient `"info_client"`, le programme plantera avec une `KeyError` au moment où le LLM demandera l'outil.

Étape 6 – La boucle de l'agent : le cœur du projet

La fonction executer_agent (agent.py, lignes 92 à 125)

```
 92  # ---------- 3. La boucle de l'agent ----------
 93  SYSTEM_PROMPT = "Tu es un assistant commercial d'Atos. Utilise les outils pour répondre. Réponds
en français."
 94
 95
 96  def executer_agent(messages):
 97      """Fait travailler l'agent sur la conversation `messages`.
 98      Renvoie la réponse finale et la liste des outils utilisés."""
 99      outils_utilises = []
100
101      for etape in range(5):  # garde-fou : 5 étapes maximum
102          reponse = client.chat.completions.create(
103              model=deployment, messages=messages, tools=DESCRIPTIONS)
104          message = reponse.choices[0].message
105
106          # Cas 1 : le LLM n'a plus besoin d'outil -> il répond
107          if not message.tool_calls:
108              messages.append({"role": "assistant", "content": message.content})
109              return message.content, outils_utilises
110
111          # Cas 2 : le LLM demande un ou plusieurs outils -> on les exécute
112          messages.append(message)
113          for appel in message.tool_calls:
114              nom_outil = appel.function.name
115              arguments = json.loads(appel.function.arguments)
116              print(f"   [outil] {nom_outil}({arguments})")
117
118              resultat = OUTILS[nom_outil](**arguments)
119              print(f"   [résultat] {resultat}")
120              outils_utilises.append({"outil": nom_outil, "arguments": arguments, "resultat":
resultat})
121
122              # On renvoie le résultat au LLM avec le rôle "tool"
123              messages.append({"role": "tool", "tool_call_id": appel.id, "content": resultat})
124
```

```
125      return "Désolé, je n'ai pas pu terminer : trop d'étapes.", outils_utilises
```

Lignes	Explication
93	Les consignes générales de l'assistant (rôle system). Elles sont dans une variable pour être réutilisées par le serveur web.
96	La boucle est rangée dans une **fonction** : le programme du terminal **et** le serveur web (partie B) l'utilisent. Elle reçoit la conversation `messages`.
99	On note les outils utilisés, pour pouvoir les afficher dans l'interface web.
101	**Garde-fou** : 5 allers-retours maximum. Sans lui, un agent qui se trompe pourrait tourner en boucle et coûter cher.
102–103	L'appel au LLM. La nouveauté par rapport aux projets précédents : `tools=DESCRIPTIONS` donne au LLM le catalogue des outils.
107–109	**Cas 1 :** le LLM ne demande aucun outil (`tool_calls` est vide). C'est sa réponse finale : on l'ajoute à l'historique et on la renvoie.
112	**Cas 2 :** le LLM demande des outils. On ajoute d'abord sa demande à l'historique : l'API exige de voir la demande avant les résultats.
113	Le LLM peut demander **plusieurs outils en même temps** : on les exécute tous.
114–115	Le nom de l'outil, et ses arguments. Le LLM les envoie sous forme de texte JSON (`{"nom": "Banque Horizon"}`) : `json.loads` les transforme en dictionnaire Python.
118	**L'exécution.** `OUTILS[nom_outil]` retrouve la fonction ; `**arguments` « déballe » le dictionnaire en arguments nommés : `infos_client(nom="Banque Horizon")`.
120	On mémorise l'outil, ses arguments et son résultat.
123	On renvoie le résultat au LLM avec le rôle `tool`. `tool_call_id` relie ce résultat à la demande correspondante. Puis la boucle repart : nouvel appel au LLM.
125	Si les 5 étapes sont épuisées, on renvoie un message clair plutôt que de planter.

6.1 Ce que contient la liste messages, étape par étape

Pour la question « Quel est le budget de Banque Horizon en dollars ? », voici l'historique à la fin du travail. C'est ce qui est renvoyé au LLM à chaque tour de boucle :

#	Rôle	Contenu
1	`system`	Tu es un assistant commercial d'Atos. Utilise les outils…
2	`user`	Quel est le budget de Banque Horizon en dollars ?
3	`assistant`	*(demande d'outil)* `infos_client(nom="Banque Horizon")`
4	`tool`	{"secteur": "Banque", "ville": "Lyon", …, "budget_euros": 4200000}
5	`assistant`	*(demande d'outil)* `convertir_euros(montant=4200000, devise="USD")`
6	`tool`	4200000 EUR = 4536000.0 USD
7	`assistant`	Le budget de Banque Horizon est de 4 200 000 €, soit 4 536 000 USD.

Trois appels au LLM ont été nécessaires (après les messages 2, 4 et 6). C'est pourquoi un agent consomme plus de tokens qu'un simple chatbot.

Étape 7 – Le programme dans le terminal

Le programme principal (agent.py, lignes 128 à 145)

```
128  # ---------- 4. Le programme dans le terminal ----------
129  # Ce bloc s'exécute avec "python agent.py", mais PAS quand app.py fait "from agent import ..."
130  if __name__ == "__main__":
131      messages = [{"role": "system", "content": SYSTEM_PROMPT}]
132
133      print("Pose une question à l'agent (tape quit pour quitter).")
134      print("Exemple : Quel est le budget de Banque Horizon en dollars ?\n")
135
136      while True:
137          question = input("Vous > ").strip()
138          if question.lower() == "quit":
139              break
140          if not question:
141              continue
142          messages.append({"role": "user", "content": question})
143
144          reponse, outils = executer_agent(messages)
145          print("\nAssistant >", reponse, "\n")
```

Lignes	Explication
130	Ce bloc ne s'exécute que si l'on lance `python agent.py`. Quand `app.py` fait `from agent import ...`, il est ignoré : on peut réutiliser l'agent sans lancer le terminal.
131	La conversation commence par les consignes.
136–141	La boucle du chat : on lit une question ; `quit` arrête ; une saisie vide est ignorée.
142	On ajoute la question à l'historique.
144	On fait travailler l'agent, puis on affiche sa réponse. L'historique s'allonge à chaque question : l'agent se souvient de la conversation.

7.1 Exécution et résultat attendu

```
python agent.py
```

```
Pose une question à l'agent (tape quit pour quitter).
Exemple : Quel est le budget de Banque Horizon en dollars ?
Vous > Quel est le budget de Banque Horizon en dollars ?
   [outil] infos_client({'nom': 'Banque Horizon'})
   [résultat] {"secteur": "Banque", "ville": "Lyon", "projet": "Migration cloud + SOC 24/7",
"budget_euros": 4200000}
   [outil] convertir_euros({'montant': 4200000, 'devise': 'USD'})
   [résultat] 4200000 EUR = 4536000.0 USD
Assistant > Le budget de Banque Horizon est de 4 200 000 €, soit environ 4 536 000 USD.
Vous > quit
```

7.2 Questions à essayer

Question	Outils attendus	Ce qu'on observe
Quel est le budget de Banque Horizon en dollars ?	infos_client → convertir_euros	Deux outils enchaînés
Quelle date sommes-nous ?	date_du_jour	Un outil sans argument
Convertis 1000 euros en francs CFA	convertir_euros (XOF)	Le LLM traduit « francs CFA » en code XOF
Et en livres sterling ?	convertir_euros (GBP)	L'agent se souvient de la question précédente

Question	Outils attendus	Ce qu'on observe
Parle-moi du client Orange	infos_client	Client inconnu : l'agent le dit au lieu d'inventer
Bonjour, qui es-tu ?	aucun	Pas besoin d'outil : réponse directe

Étape 8 – Dépannage et exercices

Symptôme	Cause probable	Solution
`KeyError: 'CHAT_DEPLOYMENT'`	.env absent ou incomplet	Créer le .env et remplir les 3 valeurs
`ModuleNotFoundError: dotenv`	Environnement virtuel non activé	Activer le .venv puis pip install
`401 Unauthorized`	Clé API incorrecte	Corriger la clé dans le .env
`404 DeploymentNotFound`	Nom de modèle au lieu du nom de déploiement	Copier le nom depuis Azure AI Foundry
`KeyError` sur un nom d'outil	Nom différent entre DESCRIPTIONS et OUTILS	Aligner les deux noms
`TypeError: ... unexpected keyword`	Paramètre décrit différemment de la fonction Python	Aligner les noms des paramètres
L'agent n'utilise pas l'outil	Description trop vague	Préciser la description de l'outil

✎ Exercice 1 – Ajouter un 4e outil

Ajoute un outil `calculer_tva(montant, taux)` qui renvoie le montant TTC.
Trois endroits à modifier : la fonction, le dictionnaire `OUTILS`, et une entrée dans `DESCRIPTIONS`.
Test : « Quel est le budget TTC d'Energia Nord avec une TVA de 20 % ? »

✎ Exercice 2 – Observer le garde-fou

Remplace `range(5)` par `range(1)` et repose la question sur le budget en dollars.
Que se passe-t-il ? Pourquoi ce garde-fou est-il indispensable en production ?

✎ Exercice 3 – Le pouvoir de la description

Remplace la description de `convertir_euros` par « Fait un calcul. » et observe si le LLM l'utilise encore.
Conclusion : la qualité d'un agent dépend beaucoup de la qualité des descriptions de ses outils.

Partie B – TP : une interface web comme ChatGPT

Objectif du TP : remplacer le terminal par une vraie page web de discussion : on tape une question, l'assistant répond dans une bulle, les outils utilisés sont affichés, et la conversation garde son historique. Le tout avec le même agent que la partie A, **sans modifier sa logique**.

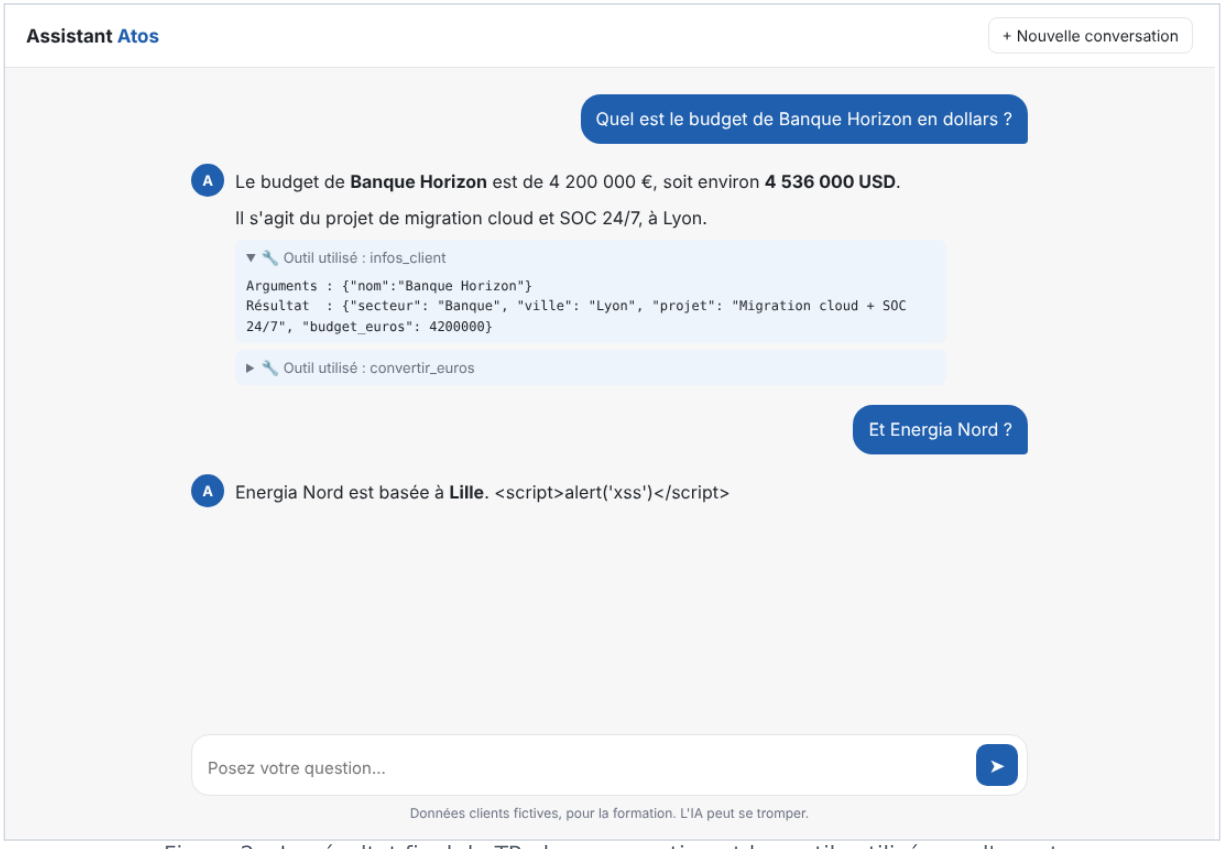


Figure 2 – Le résultat final du TP : la conversation et les outils utilisés par l'agent

Étape 9 – L'architecture : trois acteurs

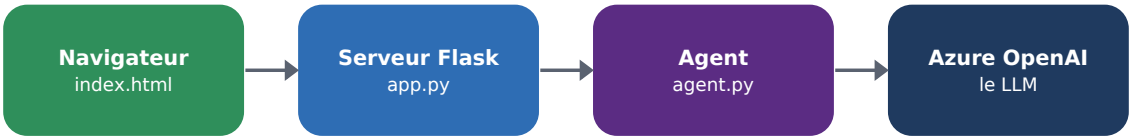


Figure 3 – Le navigateur parle au serveur Flask, qui fait travailler l'agent, qui appelle Azure OpenAI

Acteur	Langage	Rôle
Le navigateur	HTML, CSS, JavaScript	Affiche la page, lit la question, l'envoie au serveur, affiche la réponse.
Le serveur Flask	Python	Reçoit les questions sur l'adresse `/api/chat`, appelle l'agent, renvoie la réponse en JSON.
L'agent	Python	La fonction `executer_agent` de la partie A, réutilisée telle quelle.
Azure OpenAI	—	Le LLM, appelé uniquement par le serveur.

> ⚠ **Pourquoi ne pas appeler Azure OpenAI directement depuis la page web ?**
> Parce que la clé API serait visible par tout le monde : n'importe qui peut lire le code JavaScript d'une page (touche F12). **La clé reste sur le serveur**, le navigateur ne la voit jamais.

9.1 Le dialogue entre le navigateur et le serveur

Le navigateur envoie une requête **POST** sur `/api/chat` avec toute la conversation, au format JSON :

```
{
  "historique": [
    {"role": "user",      "content": "Quel est le budget de Banque Horizon en dollars ?"},
    {"role": "assistant", "content": "Le budget de Banque Horizon est de 4 200 000 €..."},
    {"role": "user",      "content": "Et en livres sterling ?"}
  ]
}
```

Le serveur répond, toujours en JSON :

```
{
  "reponse": "Cela représente 3 570 000 GBP.",
  "outils": [
    {"outil": "convertir_euros", "arguments": {"montant": 4200000, "devise": "GBP"},
     "resultat": "4200000 EUR = 3570000.0 GBP"}
  ]
}
```

> ℹ **Pourquoi le navigateur renvoie-t-il toute la conversation ?**
> Le LLM n'a pas de mémoire. Ici, c'est le **navigateur** qui garde l'historique et le renvoie à chaque question. Le serveur ne mémorise rien (il est « sans état ») : plusieurs stagiaires peuvent l'utiliser en même temps sans que leurs conversations se mélangent.

Étape 10 – Organiser les fichiers et installer Flask

```
projet2-agent/
├── .env                  # tes secrets (non commité)
├── agent.py              # l'agent (partie A)
├── app.py                # NOUVEAU : le serveur web Flask
├── requirements.txt      # + flask
└── templates/
    └── index.html        # NOUVEAU : la page de chat
```

Flask cherche les pages HTML dans un dossier qui s'appelle obligatoirement `templates`. Flask est déjà dans `requirements.txt` ; s'il manque :

```
pip install flask
```

Étape 11 – Le serveur : app.py

Fichier complet : app.py (49 lignes)

```
 1  # app.py
 2  # TP : une interface web de type ChatGPT pour notre agent.
 3  #   - le navigateur affiche la page templates/index.html
 4  #   - quand on envoie une question, la page appelle /api/chat
 5  #   - le serveur fait travailler l'agent (agent.py) et renvoie la réponse en JSON
 6  from flask import Flask, jsonify, render_template, request
 7  from openai import OpenAIError
 8
 9  from agent import SYSTEM_PROMPT, executer_agent
10
11  app = Flask(__name__)
12
13
14  # ---------- Route 1 : la page web ----------
15  @app.get("/")
16  def accueil():
17      return render_template("index.html")
18
19
20  # ---------- Route 2 : l'API de chat ----------
21  @app.post("/api/chat")
22  def chat():
23      donnees = request.get_json(silent=True) or {}
24      historique = donnees.get("historique", [])
25
26      # Le navigateur envoie toute la conversation (le LLM n'a pas de mémoire).
27      # Sécurité : on ne fait pas confiance au navigateur. On ne garde que les rôles
28      # "user" et "assistant" (pas de faux message "system"), les 20 derniers messages,
29      # et des textes de 2 000 caractères maximum.
30      messages = [{"role": "system", "content": SYSTEM_PROMPT}]
31      for message in historique[-20:]:
32          if message.get("role") in ("user", "assistant") and isinstance(message.get("content"), str):
33              messages.append({"role": message["role"], "content": message["content"][:2000]})
34
35      if messages[-1]["role"] != "user":
36          return jsonify({"erreur": "La conversation doit se terminer par une question."}), 400
37
38      try:
39          reponse, outils = executer_agent(messages)
40      except OpenAIError as erreur:
41          print("Erreur Azure OpenAI :", erreur)  # le détail reste côté serveur
42          return jsonify({"erreur": "Le service IA ne répond pas. Réessayez dans un instant."}), 502
43
44      return jsonify({"reponse": reponse, "outils": outils})
45
46
47  if __name__ == "__main__":
48      # debug=True : rechargement automatique quand on modifie le code. JAMAIS en production.
49      app.run(debug=True, port=5000)
```

Lignes	Explication
6	`Flask` crée l'application, `render_template` envoie une page HTML, `request` lit ce que le navigateur envoie, `jsonify` renvoie du JSON.
9	**On réutilise l'agent de la partie A.** Grâce au `if __name__ == "__main__"` d'agent.py, cet import ne lance pas le chat du terminal.
11	L'application web.
15–17	**Route 1.** Quand le navigateur ouvre l'adresse `/`, on renvoie la page `templates/index.html`.
21	**Route 2 : l'API.** Elle ne répond qu'aux requêtes POST envoyées par le JavaScript de la page.
23–24	On lit le JSON envoyé. `silent=True` et `or {}` évitent un plantage si le contenu est invalide.
30–33	**Sécurité.** Le navigateur n'est pas digne de confiance (on peut envoyer n'importe quoi à l'API). On reconstruit donc la conversation : notre propre message `system`, puis uniquement des messages `user`/`assistant`, les 20 derniers, de 2 000 caractères maximum. Impossible d'injecter un faux message `system`.
35–36	La conversation doit se terminer par une question. Sinon : erreur **400** (« requête invalide »).
39	On fait travailler l'agent.
40–42	Si Azure ne répond pas, on renvoie un message simple au navigateur (code **502**), et on garde le détail technique dans le terminal du serveur.
44	La réponse et la liste des outils utilisés, en JSON.
49	Démarre le serveur sur le port 5000. debug=True recharge le serveur à chaque modification du code : pratique en développement, **dangereux en production**.

Étape 12 – La page web : index.html

Une page web est composée de trois langages, chacun avec son rôle. Le fichier est découpé en trois parties dans cet ordre :

Partie	Langage	Rôle	Analogie
1	**CSS**	Le style : couleurs, tailles, disposition, thème sombre	La décoration
2	**HTML**	La structure : en-tête, zone des messages, zone de saisie	Les murs de la maison
3	**JavaScript**	Le comportement : envoyer, recevoir, afficher	L'électricité

12.1 La structure HTML

La structure de la page (index.html, lignes 115 à 141)

```
115    <!-- ==================== 2. LA STRUCTURE (HTML) ==================== -->
116    <header>
117      <h1>Assistant <span>Atos</span></h1>
118      <button id="nouvelle" type="button">+ Nouvelle conversation</button>
119    </header>
120
121    <main id="zone">
122      <div id="conversation">
123        <div id="accueil">
124          <h2>Bonjour □</h2>
125          <p>Je suis l'assistant commercial d'Atos. Posez-moi une question sur nos clients.</p>
126          <div id="suggestions">
127            <button type="button">Quel est le budget de Banque Horizon en dollars ?</button>
128            <button type="button">Dans quelle ville est Energia Nord ?</button>
129            <button type="button">Convertis 1000 euros en francs CFA</button>
130            <button type="button">Quelle date sommes-nous ?</button>
131          </div>
132        </div>
```

```
133      </div>
134    </main>
135
136    <footer>
137      <form id="formulaire">
138        <textarea id="question" rows="1" placeholder="Posez votre question..." maxlength="2000"></
textarea>
139        <button id="envoyer" type="submit" title="Envoyer">➤</button>
140      </form>
141      <div class="note">Données clients fictives, pour la formation. L'IA peut se tromper.</div>
```

Lignes	Explication
116–119	L'en-tête : le titre et le bouton « Nouvelle conversation » (`id="nouvelle"`).
121	La zone qui défile. Elle contient `#conversation`, où le JavaScript ajoutera les messages.
123–126	L'écran d'accueil, masqué dès le premier message.
127–130	Des questions toutes prêtes : un clic les envoie. Pratique en démonstration.
137–140	La zone de saisie : un `textarea` (texte sur plusieurs lignes, `maxlength="2000"` comme côté serveur) et le bouton d'envoi.

12.2 Le style CSS (les points importants)

Les couleurs (index.html, lignes 8 à 17)

```
 8    <!-- ==================== 1. LE STYLE (CSS) ==================== -->
 9    <style>
10      :root {
11        --fond: #f7f7f8;          --carte: #ffffff;
12        --texte: #1f2328;         --texte-doux: #6b7280;
13        --bordure: #e5e7eb;       --accent: #1f5fae;
14        --bulle-user: #1f5fae;    --texte-user: #ffffff;
15        --outil: #eef4fb;
16      }
17      @media (prefers-color-scheme: dark) {   /* thème sombre automatique */
```

Lignes	Explication
10	Les **variables CSS** (`--accent`, `--fond`…) : toutes les couleurs sont définies à un seul endroit. Changer la couleur de la marque = changer une ligne.
17	Le **thème sombre automatique** : si l'ordinateur est en mode sombre, les variables prennent d'autres valeurs.
26	La page occupe tout l'écran (`height: 100vh`) et empile en colonne l'en-tête, les messages et la saisie (`display: flex`).
47	La zone des messages prend toute la place restante et défile (`overflow-y: auto`).
58	Les bulles de l'utilisateur : fond bleu, coins arrondis, alignées à droite.
76–82	Les trois points animés « l'assistant réfléchit ».

12.3 Le JavaScript, bloc par bloc

Les éléments et l'historique (index.html, lignes 146 à 155)

```
146      // --- 3.1 Les éléments de la page dont on a besoin ---
147      const conversation = document.getElementById("conversation");
148      const zone = document.getElementById("zone");
149      const formulaire = document.getElementById("formulaire");
150      const champ = document.getElementById("question");
151      const boutonEnvoyer = document.getElementById("envoyer");
152      const accueil = document.getElementById("accueil");
153
154      // --- 3.2 L'historique de la conversation (renvoyé au serveur à chaque question) ---
```

```
155     let historique = [];
```

`document.getElementById` retrouve un élément de la page par son `id`. `historique` est la mémoire de la conversation côté navigateur : la liste envoyée au serveur.

La mise en forme et la sécurité (index.html, lignes 157 à 170)

```
157     // --- 3.3 Mise en forme du texte ---
158     // échapper : empêche qu'une réponse contenant du HTML soit exécutée (sécurité, faille XSS)
159     function echapper(texte) {
160       const div = document.createElement("div");
161       div.textContent = texte;
162       return div.innerHTML;
163     }
164     // mettreEnForme : **gras** et paragraphes, après avoir échappé le texte
165     function mettreEnForme(texte) {
166       return echapper(texte)
167         .replace(/\*\*(.+?)\*\*/g, "<strong>$1</strong>")
168         .split(/\n{2,}/)
169         .map(paragraphe => "<p>" + paragraphe.replace(/\n/g, "<br>") + "</p>")
170         .join("");
```

Lignes	Explication
159–162	**Sécurité (faille XSS).** Si une réponse contenait `<script>`, l'insérer telle quelle exécuterait du code dans la page. `textContent` transforme les caractères spéciaux (< devient `<`) : le texte est affiché, jamais exécuté.
165	D'abord on échappe, **ensuite** on met en forme.
167	Une expression régulière transforme `**texte**` (gras en Markdown, souvent utilisé par les LLM) en gras HTML.
168–170	Les lignes vides séparent les paragraphes ; les retours à la ligne simples deviennent ` `.

Afficher un message (index.html, lignes 173 à 194)

```
173     // --- 3.4 Afficher un message dans la page ---
174     function afficherMessage(role, texte, outils = []) {
175       accueil.style.display = "none";
176       const div = document.createElement("div");
177       div.className = "message " + role;
178
179       let html = role === "user" ? "" : '<div class="avatar">A</div>';
180       html += '<div class="bulle">' + mettreEnForme(texte);
181       if (outils.length > 0) {
182         html += '<div class="outils">';
183         for (const o of outils) {
184           html += "<details><summary>🔧 Outil utilisé : " + echapper(o.outil) + "</summary>"
185                + "<pre>Arguments : " + echapper(JSON.stringify(o.arguments)) + "\n"
186                + "Résultat  : " + echapper(o.resultat) + "</pre></details>";
187         }
188         html += "</div>";
189       }
190       html += "</div>";
191
192       div.innerHTML = html;
193       conversation.appendChild(div);
194       zone.scrollTop = zone.scrollHeight;   // descendre en bas de la conversation
```

Lignes	Explication
174	Crée une bulle pour un rôle (`user` ou `assistant`), avec, si besoin, la liste des outils.
175	On masque l'écran d'accueil.
160–177	On fabrique un nouvel élément HTML.
181	Pour chaque outil : un bloc dépliable `<details>` avec ses arguments et son résultat. **Chaque valeur est échappée**.

Lignes	Explication
193	On ajoute la bulle à la conversation…
194	…et on fait défiler jusqu'en bas.

Envoyer une question au serveur (index.html, lignes 204 à 234)

```
204     // --- 3.5 Envoyer une question au serveur ---
205     async function envoyer(question) {
206       historique.push({ role: "user", content: question });
207       afficherMessage("user", question);
208       champ.value = "";
209       ajusterHauteur();
210       boutonEnvoyer.disabled = true;
211       const attente = afficherPoints();
212
213       try {
214         const reponse = await fetch("/api/chat", {
215           method: "POST",
216           headers: { "Content-Type": "application/json" },
217           body: JSON.stringify({ historique: historique })
218         });
219         const donnees = await reponse.json();
220         attente.remove();
221
222         if (!reponse.ok) {
223           historique.pop();   // la question n'a pas eu de réponse : on la retire
224           afficherMessage("assistant erreur", donnees.erreur || "Erreur inconnue.");
225         } else {
226           historique.push({ role: "assistant", content: donnees.reponse });
227           afficherMessage("assistant", donnees.reponse, donnees.outils);
228         }
229       } catch (e) {
230         attente.remove();
231         historique.pop();
232         afficherMessage("assistant erreur", "Impossible de joindre le serveur. Est-il démarré ?");
233       }
234       boutonEnvoyer.disabled = false;
```

Lignes	Explication
205	Fonction **asynchrone** : elle attend la réponse du serveur (`await`) sans bloquer la page.
206–207	On ajoute la question à l'historique et on l'affiche tout de suite.
210–211	On désactive le bouton (pas de double envoi) et on affiche les trois points.
214–217	**L'appel au serveur.** `fetch` envoie une requête POST à `/api/chat` avec l'historique converti en JSON (`JSON.stringify`).
219–220	On lit la réponse JSON et on retire les trois points.
222–224	Erreur renvoyée par le serveur (400, 502) : on retire la question restée sans réponse de l'historique, et on affiche l'erreur en rouge.
226–227	Succès : on mémorise la réponse et on l'affiche avec ses outils.
229	Le serveur est injoignable (arrêté, planté) : message explicite.

Les événements (index.html, lignes 238 à 264)

```
238     // --- 3.6 Les événements (clics, touches) ---
239     formulaire.addEventListener("submit", event => {
240       event.preventDefault();               // empêche le rechargement de la page
241       const question = champ.value.trim();
242       if (question && !boutonEnvoyer.disabled) envoyer(question);
243     });
244
245     champ.addEventListener("keydown", event => {
```

```
246       if (event.key === "Enter" && !event.shiftKey) {   // Entrée = envoyer, Maj+Entrée = retour à
la ligne
247         event.preventDefault();
248         formulaire.requestSubmit();
249       }
250     });
251
252     function ajusterHauteur() {             // la zone de saisie grandit avec le texte
253       champ.style.height = "auto";
254       champ.style.height = champ.scrollHeight + "px";
255     }
256     champ.addEventListener("input", ajusterHauteur);
257
258     document.querySelectorAll("#suggestions button").forEach(bouton =>
259       bouton.addEventListener("click", () => envoyer(bouton.textContent)));
260
261     document.getElementById("nouvelle").addEventListener("click", () => {
262       historique = [];
263       conversation.querySelectorAll(".message").forEach(m => m.remove());
264       accueil.style.display = "";
```

Lignes	Explication
239	Clic sur le bouton d'envoi. `event.preventDefault()` empêche le comportement par défaut d'un formulaire (recharger la page).
245	**Entrée** envoie la question, **Maj + Entrée** fait un retour à la ligne, comme dans ChatGPT.
252	La zone de saisie grandit avec le texte.
258	Un clic sur une suggestion envoie son texte.
261	« Nouvelle conversation » : on vide l'historique et les bulles, et on réaffiche l'accueil. L'agent repart de zéro.

Étape 13 – Lancer et tester l'application

13.1 Démarrer le serveur

```
python app.py
```

```
 * Serving Flask app 'app'
 * Debug mode: on
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
```

Ouvre **http://127.0.0.1:5000** dans le navigateur. Le terminal doit rester ouvert : c'est le serveur. Pour l'arrêter : `Ctrl+C`.

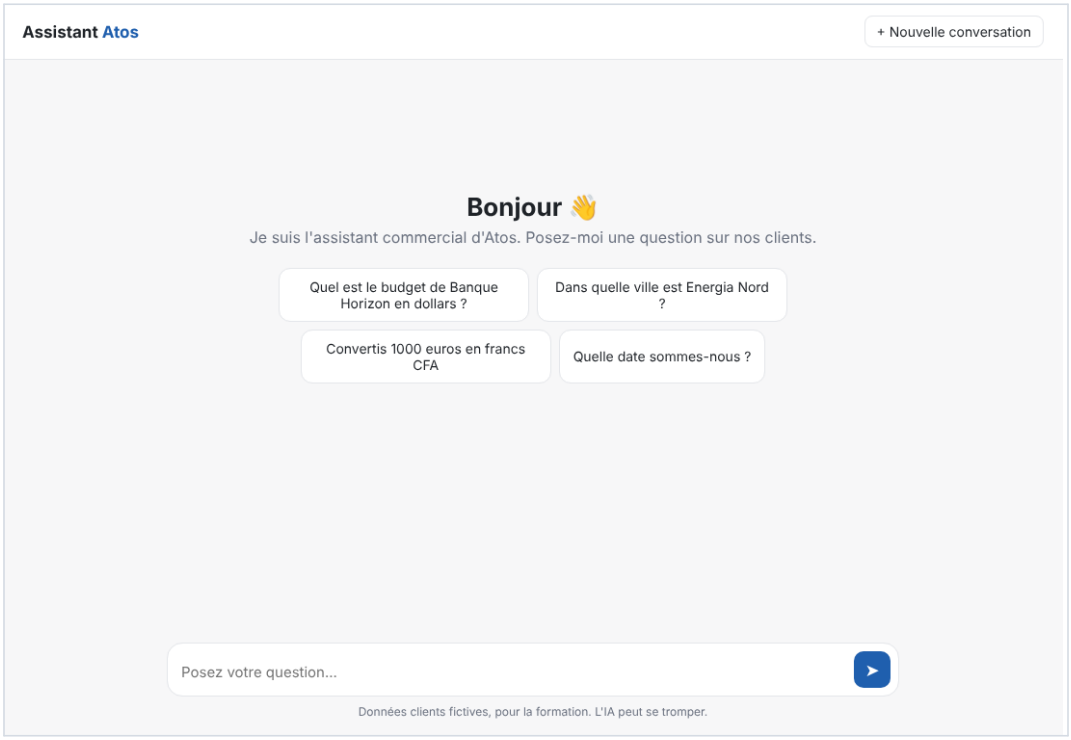


Figure 4 – L'écran d'accueil avec les questions suggérées

13.2 Scénario de test

1. Cliquer sur la suggestion « Quel est le budget de Banque Horizon en dollars ? ».
2. Vérifier : les trois points s'affichent, puis la réponse et deux blocs « Outil utilisé ».
3. Déplier un bloc d'outil : les arguments et le résultat sont visibles.
4. Taper « Et en livres sterling ? » puis Entrée : l'agent comprend grâce à l'historique.
5. Regarder le terminal du serveur : les lignes [outil] et [résultat] s'y affichent aussi.
6. Cliquer sur « Nouvelle conversation », puis demander « Et en livres sterling ? » : l'agent ne sait plus de quoi on parle. Preuve que la mémoire est dans l'historique.
7. Arrêter le serveur (Ctrl+C) et envoyer une question : le message « Impossible de joindre le serveur » s'affiche.

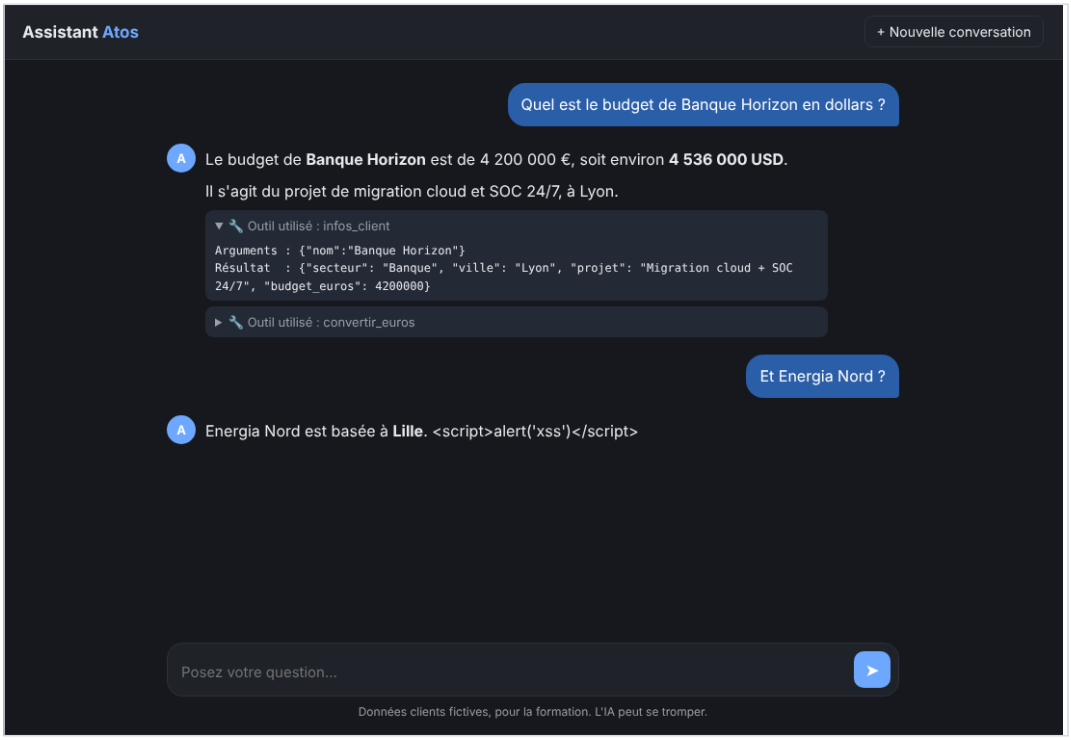


Figure 5 – Le thème sombre s'active automatiquement selon le réglage de l'ordinateur

13.3 Observer les échanges avec les outils du navigateur

1. Appuyer sur **F12** (outils de développement), onglet **Réseau** (Network).
2. Envoyer une question : une ligne `chat` apparaît.
3. Cliquer dessus : l'onglet **Charge utile** (Payload) montre l'historique envoyé, l'onglet **Réponse** montre le JSON renvoyé par le serveur.
4. Remarque à faire aux stagiaires : la clé API n'apparaît nulle part.

13.4 Tester l'API sans navigateur

On peut appeler l'API directement, par exemple depuis PowerShell :

```
Invoke-RestMethod -Method Post -Uri http://127.0.0.1:5000/api/chat `
  -ContentType "application/json" `
  -Body '{"historique":[{"role":"user","content":"Quelle date sommes-nous ?"}]}'
```

Sous macOS / Linux :

```
curl -X POST http://127.0.0.1:5000/api/chat -H "Content-Type: application/json" \
  -d '{"historique":[{"role":"user","content":"Quelle date sommes-nous ?"}]}'
```

Étape 14 – Dépannage du TP

Symptôme	Cause probable	Solution
`ModuleNotFoundError: flask`	Flask non installé dans le .venv	`pip install -r requirements.txt`
`TemplateNotFound: index.html`	Le fichier n'est pas dans un dossier `templates`	Ranger index.html dans `templates/`
`Address already in use`	Le port 5000 est déjà utilisé	`app.run(port=5001)` et ouvrir le port 5001

Symptôme	Cause probable	Solution
La page ne s'ouvre pas	Le serveur n'est pas démarré ou a planté	Lire les erreurs dans le terminal
« Impossible de joindre le serveur »	Serveur arrêté pendant l'utilisation	Relancer `python app.py`
« Le service IA ne répond pas »	Erreur Azure (clé, déploiement, quota)	Le détail est dans le terminal du serveur
La page ne change pas après une modification	Le navigateur a gardé l'ancienne version	Recharger avec `Ctrl+F5`

Étape 15 – Pour aller plus loin, et évaluation du TP

15.1 Améliorations possibles (exercices)

Niveau	Amélioration	Indice
★	Changer les couleurs et le nom de l'assistant	Variables CSS dans `:root`
★	Ajouter une 5e suggestion	Un `<button>` de plus dans `#suggestions`
★★	Afficher le temps de réponse sous chaque réponse	`Date.now()` avant et après `fetch`
★★	Ajouter un bouton « Copier » sur chaque réponse	`navigator.clipboard.writeText(...)`
★★	Ajouter les protections du module M4 (longueur, injection, limite de débit)	Réutiliser `verifier_entree` et `LimiteurDeDebit` dans app.py
★★★	Afficher la réponse mot par mot (streaming)	`stream=True` côté Python, lecture du flux côté JavaScript
★★★	Garder les conversations après un rechargement	`localStorage` dans le navigateur
★★★	Déployer sur Azure App Service	Serveur de production (gunicorn) + identité managée au lieu de la clé

> **⚠ Avant une mise en production**
> Désactiver `debug=True` et utiliser un vrai serveur (gunicorn, Azure App Service).
> Ajouter une authentification (par exemple Entra ID) : aujourd'hui, toute personne qui accède à l'adresse peut utiliser l'assistant, et donc le budget Azure.
> Appliquer la checklist d'audit en 20 points du module M4.

15.2 Grille d'évaluation du TP (sur 20)

Critère	Points
Le serveur démarre et la page s'affiche	3
Une question reçoit une réponse de l'agent dans une bulle	4
Les outils utilisés sont affichés sous la réponse	3
L'historique fonctionne (question de suivi comprise)	3
« Nouvelle conversation » remet l'agent à zéro	2
Les erreurs sont gérées (serveur arrêté, erreur Azure)	2

Critère	Points
Sécurité : texte échappé (XSS), clé jamais côté navigateur, rôles filtrés	2
Une amélioration de la liste 15.1 réalisée	1

Annexes

Annexe 1 – Récapitulatif des commandes

```
python -m venv .venv
.venv\Scripts\Activate.ps1          # macOS/Linux : source .venv/bin/activate
python -m pip install --upgrade pip
pip install -r requirements.txt
Copy-Item .env.example .env           # macOS/Linux : cp .env.example .env
python agent.py                       # Partie A : l'agent dans le terminal (quit pour quitter)
python app.py                         # Partie B : l'interface web -> http://127.0.0.1:5000 (Ctrl+C
pour arrêter)
```

Annexe 2 – Glossaire

Terme	Définition
API	Une adresse qu'un programme appelle pour obtenir un service (ici /api/chat).
Agent	Un LLM doté d'outils, qui décide seul des actions à enchaîner pour répondre.
Asynchrone (async/await)	Attendre un résultat (une réponse réseau) sans bloquer le reste du programme.
CSS	Le langage du style d'une page web.
Déploiement	L'instance d'un modèle créée dans Azure, avec un nom choisi : c'est ce nom qu'on passe à model=.
fetch	La fonction JavaScript qui envoie une requête HTTP au serveur.
Flask	Un mini framework Python pour créer un serveur web.
Function calling	Le mécanisme par lequel le LLM demande l'exécution d'une fonction décrite en JSON.
HTML	Le langage de la structure d'une page web.
JSON	Un format texte d'échange de données : {"cle": "valeur"}.
JavaScript	Le langage qui donne un comportement à une page web, exécuté par le navigateur.
Route	Une adresse gérée par le serveur Flask (/ ou /api/chat).
Sans état (stateless)	Un serveur qui ne mémorise rien entre deux requêtes.
XSS	Faille qui consiste à faire exécuter du code JavaScript dans la page d'un autre utilisateur.